ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГАОУ ВО НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет компьютерных наук
Образовательная программа «Прикладная математика и информатика»

Отчет о программном проекте

на тему: 3D Render с нуля

Выполнил:
Студент группы БПМИ215 _____ А.В.Проскурин
Подпись И.О.Фамилия

13.05.2024
Дата

Принял:
Руководитель проекта Лукьяненко Никита Сергеевич
Имя, Отчество, Фамилия
старший преподаватель, к.ф.-м.н.
Должность, ученое звание
ДБДИП ФКН НИУ ВШЭ
Место работы (Компания или подразделение НИУ ВШЭ)

Дата проверки _____ 2024 _____ _____
Оценка (по 10-ти бальной шкале) Подпись

Москва 2024

Содержание

1 Введение **2**

2 Функциональные требования **3**

3 Нефункциональные требования **3**

4 Процесс отрисовки изображения **3**
4.1 Определения . . . 3
4.2 Пайплайн . . . 4
4.3 Переход в пространство камеры . . . 5
4.4 Clipping . . . 5
4.5 Преобразование пирамиды зрения . . . 6
4.6 Алгоритм Z-буффера . . . 7
4.7 Интерполяция скалярных величин . . . 8

5 Архитектура приложения **9**
5.1 Ключевые классы . . . 9
5.2 Вспомогательные классы . . . 10

6 Демонстрация результатов **11**

Аннотация

Цель данного проекта – познакомиться с процессом отрисовки 3D-объектов на экране, разобраться в стоящей за этим математике, а также реализовать свой интерактивный 3D renderer с нуля на языке C++.

1 Введение

Хотя мы давно привыкли к трехмерной графике в играх, сама по себе задача отрисовки трехмерных объектов на экране не является тривиальной. Она состоит и из чисто математических задач, таких как проецирования трехмерных объектов на плоскость с учетом законов перспективы и разрешения видимости объектов, и физических, таких как симуляция распространения света с учетом физических свойств поверхности, и вычислительных, чтобы полученный рендер был пригоден для приложений в реальном времени.

Целью данного проекта является разобраться во внутреннем устройстве 3D-рендеров, а также разработать свой рендер с нуля, без использования готовых библиотек, таких как OpenGL. Для демонстрации работы рендера будет разработано интерактивное приложение, позволяющее пользователю перемещаться по сцене и рассматривать ее объекты с разных ракурсов.

Далее отчет выстроен следующим образом. В продолжении секции 1 формально перечислены задачи проекта, описаны источники информации и указана ссылка на репозиторий с кодом проекта. В секциях 2 и 3 описываются функциональные и нефункциональные требования к проекту. В секции 4 описывается процесс отрисовки изображения: сначала вводятся необходимые определения и описывается пайплайн целиком, а потом подробно разбирается каждый из этапов пайплайна. В секции 5 описывается дизайн реализованного приложения, основные классы и методы. И наконец в секции 6 представлены примеры работы рендера.

Задачи проекта

1. Изучить и изложить теорию, стоящую за отрисовкой 3D объектов на экране
2. Изучить пайплайны дли отрисовки изображения и для создания интерактивного приложения
3. Реализовать интерактивное приложение на языке C++, демонстрирующее работу 3D рендерера
4. Изложить в отчете архитектуру и дизайн имплементации

Источники информации Главным источником информации является книга "Mathematics for 3D Game Programming and Computer Graphics, Third Edition"[10] – оттуда была взята вся необходимая теория по отрисовке изображения на экране. Архитектура реализации интерактивного приложения будет основываться на консультациях к проекту: [11] [12].

Ссылка на репозиторий GitHub

https://github.com/K1ngNothing/3D-Render-from-Scratch/tree/main-dev

2 Функциональные требования

Требуется создать интерактивное приложение, демонстрирующее работу рендерера в реальном времени. Данное приложение должно предоставлять пользователю возможность:

1. Создать сцену: пользователь может задать объекты как набор полигонов и задать цвет или текстуры для них.
2. Свободно двигать камеру с помощью клавиатуры, позволяя тем самым рассмотреть объекты с разных ракурсов.

3 Нефункциональные требования

1. Язык программирования: C++20, компилятор – GCC [4]
2. Система контроля версий: Git (GitHub [5])
3. Система сборки: CMake [2]
4. Style guide: модифицированный Google C++ Style Guide [6]
5. Программа форматирования кода: clang format [1]
6. Библиотеки для работы с матрицами: Eigen [3]
7. Библиотека для работы с системными вызовами и мультимедиа: SFML [7]
8. Среда разработки: Visual Studio Code [9]

4 Процесс отрисовки изображения

4.1 Определения

Прежде чем переходить к описанию процесса отрисовки изображения, необходимо ввести ряд определений:

- *Сцена*, или *мир* – множество всего, что потенциально может быть отрисовано на экране. Элементы данного множества будем называть *объектами отрисовки* или просто *объектами*.

  У мира и у каждого объекта есть свои системы координат ($\mathbb{R}^3$ со стандартным скалярным произведением $(x, y) = x^T y$). Поэтому для каждого объекта задается преобразование, переводящее данный объект в мировые координаты. Про это следует думать так: сначала мы моделируем объект в своей системе координат, а потом задаем его положение и ориентацию в системе координат мира с помощью параллельного переноса и матрицы перехода (подробнее о том, как это делается на практике, в разд. 4.3).

- *Примитив отрисовки* – элементарный объект, с которым рендер умеет работать напрямую. Любой объект отрисовки декомпозируется на примитивы отрисовки, с которыми потом работает рендер. В имплементированном рендере примитивом отрисовки является треугольник, хотя в других библиотеках могут быть и другие примитивы (в OpenGL, например, в примитивы также входят точки, отрезки, полигоны, и другие фигуры).

  Примитив отрисовки содержит информацию о его положении в пространстве, а также вспомогательную информацию для отрисовки (цвет, свойства поверхности, текстурные координаты и т.д.). Данная информация хранится только в вершинах треугольника – для получения значения характеристик в произвольной точке, используют линейную интерполяцию (подробнее в разд. 4.7).

- *Камера*, или *пирамида зрения* – усеченная пирамида, содержащая все, что пользователь видит в сцене (рис. 11 слева). Задается точкой обзора C, ближней гранью n и дальней гранью f. Такая форма камеры объясняется тем, что она описывает ту часть пространства, что будет видна, если смотреть из точки C через окно n, с ограничением дальности видимости f.

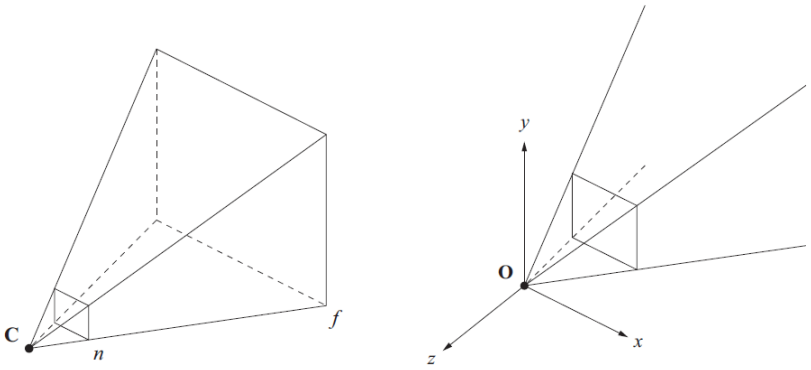


Рис. 1: (ист.: [10], Fig. 5.8 и Fig. 5.9)

В своем пространстве камера располагается как на рис. 11 справа. Чтобы учесть реальное положение камеры, мы также, как и в случае с объектами отрисовки, поддерживаем матрицу линейного преобразования, только с небольшим отличием. Эта матрица переводит из мирового пространства в пространство камеры (в отличие от матрицы объекта, переводящая в обратную сторону). Это объясняется тем, что в процессе отрисовки мы хотим перевести все примитивы отрисовки из пространств объектов в пространство камеры, для чего мы сначала переводим их в мировое пространство, а потом в пространство камеры.

4.2 Пайплайн

Пайплайн отрисовки изображения состоит из многих этапов. В этой секции мы перечислим эти этапы и кратко опишем их суть, а в последующих мы подробно остановимся на каждом из них.

Этапы отрисовки изображения:

1. Переводим все примитивы отрисовки в пространство камеры.
2. Каждый примитив отрисовки мы пересекаем с усеченной пирамидой камеры. Данный процесс называется *clipping* и он преследует 2 цели. Во-первых, это просто необходимо для корректности дальнейших преобразований. Во-вторых, это экономит ресурсы процессора / видеокарты, т.к. помогает отсечь треугольники, которые мы заведомо не можем увидеть на экране.
3. Мы применяем аффинное преобразование, переводящее усеченную пирамиду камеры в куб с координатами $x, y, z \in [-1; 1]$ (см. рис. 2). Полученное пространство мы будем называть *однородным пространством камеры* (homogeneous clip space)

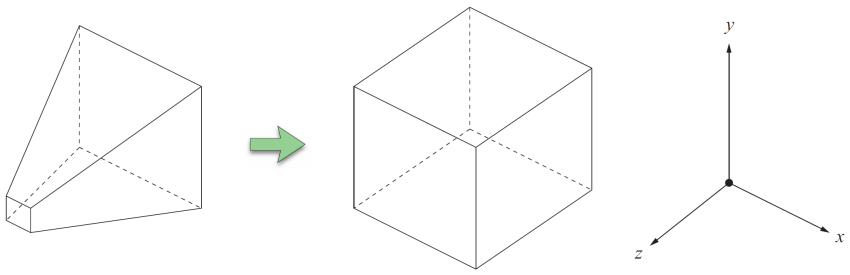


Рис. 2: Преобразование усеченной пирамиды в куб. Оси добавлены для лучшего понимания их ориентации и не отображают положение начала координат (ист.: [10], Fig. 5.15)

Данное преобразование делается для упрощения последующего проецирования примитивов отрисовки на экран (достаточно просто отбросить z-координату). Оно также позволяет учесть закон перспективы: треугольники, находящиеся ближе к экрану, будут растянуты больше тех, что расположены дальше.

4. Теперь для каждого пикселя на экране нам необходимо понять, какой из треугольников ближе всего к глазу в данном пикселе. Делать мы это будем с помощью алгоритма **Z-буффера**. Идея в следующем: т.к. ось z направлена от глаза (см. рис. 11), то чем ближе треугольник к глазу, тем меньше z-координата его точек. Поэтому мы создадим z-буффер, в котором для каждого пикселя на экране мы будем хранить минимальную z-координату точки треугольника в этом пикселе. Теперь нам достаточно рассмотреть все треугольники в сцене, вычислить пиксели, покрываемые каждым треугольником, и для каждого пикселя найти треугольник с минимальной z-координатой.

5. Последний этап: для каждого пикселя определить его цвет. Данный процесс основывается на том, что все необходимые характеристики для отрисовки треугольника задаются в его вершинах. Значит, нам достаточно сделать *интерполяцию* этих характеристик по поверхности треугольника, обычно линейную (например, если интересующей нас пиксель находится в центре треугольника, то его текстурные координаты будут равны среднему арифметическому текстурных координат в вершинах треугольника). Найдя значение характеристик треугольника в пикселе, мы вычисляем его цвет и рисуем его на экране. Проделав эту операцию для каждого пикселя, мы получаем готовое изображение.

Теперь мы подробно разберем каждый из перечисленных этапов.

4.3 Переход в пространство камеры

На первом этапе отрисовки мы переводим примитивы отрисовки в пространство камеры. Как мы упомянули в разд. 4.1, в нашей модели есть несколько независимых систем координат $\mathbb{R}^3$ со стандартным скалярным произведением: мировые координаты, координаты объектов отрисовки, координаты камеры. Соответственно, нам нужно научиться задавать преобразования, последовательно переводящие из пространства объекта $\to$ мировые координаты $\to$ пространство камеры.

Известно, что произвольное вращение точки $P \in \mathbb{R}^3$ можно задать с помощью ортогональной собственной матрицы $A \in \mathrm{M}_{3\,3}(\mathbb{R})$ следующим образом: $P \mapsto AP$, а параллельный перенос на вектор $b \in \mathbb{R}^3$: $P \mapsto P + b$. Значит, мы могли бы просто хранить матрицу A и вектор b и делать переход между координатами как преобразование $P \mapsto AP + b$. Но на практике это не очень удобно и есть способ лучше.

Перейдем в четырехмерное пространство, называемое *однородными координатами*: каждой точке $P(x, y, z)$ сопоставим точку $P'(x, y, z, w = 1)$. Тогда преобразование $P \mapsto AP + b$ можно задать умножением на матрицу $Q \in \mathrm{M}_{4\,4}(\mathbb{R})$ специального вида:

$$QP' = \left(\begin{array}{c|c} \mathbf{A} & \mathbf{b} \\ \hline \mathbf{0} & 1 \end{array}\right) \cdot \begin{pmatrix} P_x \\ P_y \\ P_z \\ \hline 1 \end{pmatrix} = \left(\begin{array}{ccc|c} a_{11} & a_{12} & a_{13} & b_1 \\ a_{21} & a_{22} & a_{23} & b_2 \\ a_{31} & a_{32} & a_{33} & b_3 \\ \hline 0 & 0 & 0 & 1 \end{array}\right) \cdot \begin{pmatrix} P_x \\ P_y \\ P_z \\ \hline 1 \end{pmatrix} = \begin{pmatrix} AP + b \\ \hline 1 \end{pmatrix}$$

Обращу внимание, что у результирующей точки w-координата равна 1, а значит данное преобразование сохраняет нашу модель. В частности, мы без проблем можем применить несколько таких преобразований подряд путем перемножения матриц преобразований.

Но у данного подхода есть побочный эффект: теперь мы обязаны разделять между собой точки и вектора. Действительно, если мы вектору сдвига $a(x, y, z) \in \mathbb{R}^3$ сопоставим новый вектор $a'(x, y, z, 1) \in \mathbb{R}^4$, то при параллельном переносе на вектор $b(x', y', z')$ мы бы получили вектор $a'_{new}(x + x', y + y', z + z', 1)$, что очевидно некорректно. Поэтому на самом деле мы должны сопоставить вектору a вектор $a'(x, y, z, 0)$. В таком случае, во-первых, параллельный перенос не будет влиять на вектор, и, во-вторых, мы все еще сможем корректно складывать вектор с вектором и точку с вектором (координата w будет 0 и 1 соответственно, как мы и ожидаем).

4.4 Clipping

Алгоритм клиппинга основывается на алгоритме Sutherland–Hodgman [8]. Идея в следующем: рассмотрим плоскости, ограничивающие пирамиду зрения, и будем пересекать примитив отрисовки с каждой плоскостью по отдельности. Т.к. мы теперь работаем с плоскостью, то нам легко понять, когда она пересекает сторону треугольника: плоскость пересекает отрезок $AB \Leftrightarrow A, B$ лежат в разных полупространствах относительно

данной плоскости. Из-за чего получается простой алгоритм, в котором мы последовательно рассматриваем вершины треугольника, и проверяем их взаимное расположение относительно плоскости:

```
for (int i = 0; i < inputList.count; i += 1) do
    Point current_point = inputList[i];
    Point prev_point = inputList[(i - 1) % inputList.count];

    Point Intersecting_point = ComputeIntersection(prev_point, current_point, clipEdge)

    if (current_point inside clipEdge) then
        if (prev_point not inside clipEdge) then
            outputList.add(Intersecting_point);
        end if
        outputList.add(current_point);

    else if (prev_point inside clipEdge) then
        outputList.add(Intersecting_point);
    end if

done
```

Рис. 3: Псевдокод одной итерации алгоритма Sutherland–Hodgman (ист.: [8])

Замечу, что после данного процесса мы не обязаны получить треугольник. Это не проблема между итерациями алгоритма (просто на вход очередной итерации будет подаваться многоугольник вместо треугольника), но после него результирующий многоугольник следует разделить на треугольники. В нашей реализации интерактивного приложения это достигается путем разбиения многоугольника диагоналями, проведенными из первой вершины во все остальные.

4.5 Преобразование пирамиды зрения

После клиппинга мы работаем только с треугольниками, полностью содержащимися внутри пирамиды зрения. Теперь для перехода от пирамиды зрения к кубу нам достаточно умножить однородные координаты вершин треугольника на матрицу специального вида:

$$\mathbf{P}' = \mathbf{M}_{\text{frustum}}\mathbf{P} = \begin{bmatrix} \frac{2n}{r-l} & 0 & \frac{r+l}{r-l} & 0 \\ 0 & \frac{2n}{t-b} & \frac{t+b}{t-b} & 0 \\ 0 & 0 & -\frac{f+n}{f-n} & -\frac{2nf}{f-n} \\ 0 & 0 & -1 & 0 \end{bmatrix} \begin{bmatrix} P_x \\ P_y \\ P_z \\ 1 \end{bmatrix}$$

Рис. 4: Матрица преобразование усеченной пирамиды в куб [1](ист.: [10], Eq. 5.52)

(обозначения: n, f – расстояния до ближнего и дальнего сонований пирамиды;
l, r, t, b – границы координат точек на ближнем основании пармиды: $l \leq x \leq r, b \leq y \leq t$)

Замечу, что полученная точка P' в однородных координатах имеет $P'_w = -P_z$. Поэтому для получения соответствующей точки в трехмерном пространстве нам необходимо разделить $P'_{x,y,z}$ на P'_w.

[1]Вывод данной матрицы можно узнать в [10], sec. 5.5

Подчеркну, что данное преобразование не будет корректным без предварительного клиппинга. Для иллюстрации можно посмотреть на то, как преобразуется $z-$координата при умножении на $\mathbf{M_{frustum}}$:

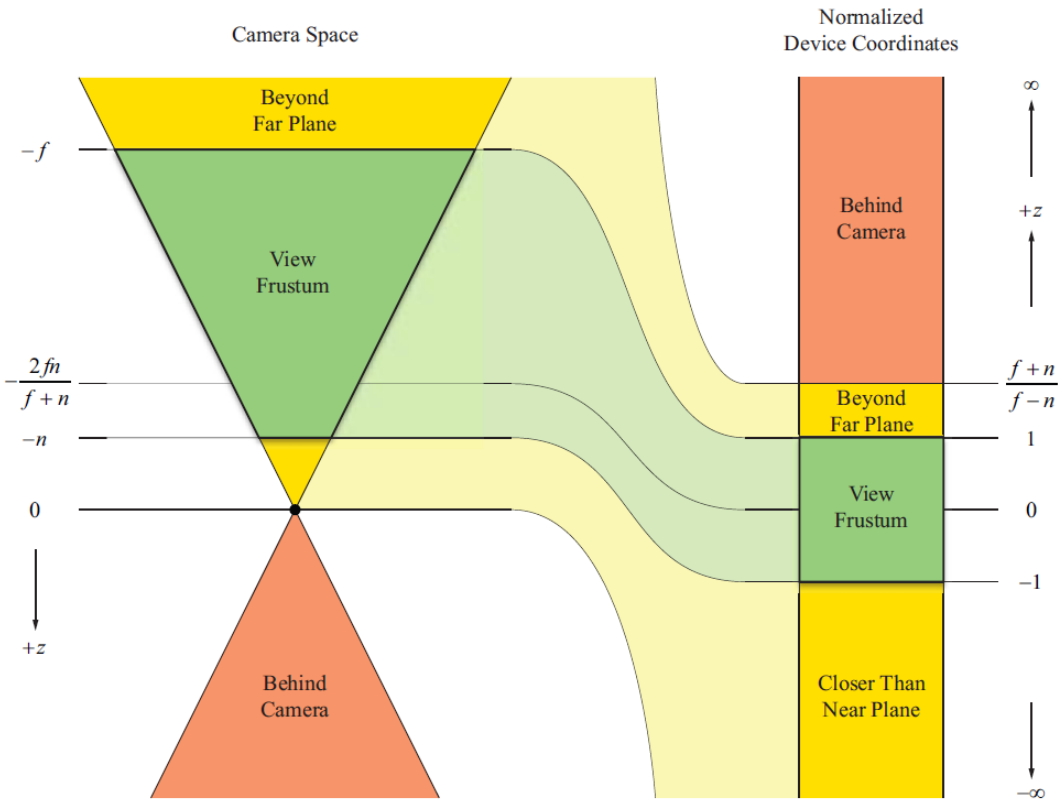


Рис. 5: Преобразование $z-$координаты под действием $\mathbf{M_{frustum}}$ (ист.: [10], Fig. 5.16)

Нетрудно видеть, что пирамида зрения преобразуется как мы и ожидаем, тогда как пространства перед и за ней ведут себя неочевидным образом (например, бесконечное пространство за камерой умещается в конечный отрезок $[1; (f+n)/(f-n)]$).

Замечу также, что в приведенной матрице можно устремить $f \to \inf$. Такое преобразование позволяет снять ограничение на дальность видимости и отрисовывать объекты на произвольном расстоянии. В таком преобразовании также можно положить $P_w = 0$ – это соответствует бесконечно удаленной точке.

4.6 Алгоритм Z-буффера

Алгоритм Z-буффера заключается в следующем: сначала мы создаем Z-буффер по размеру экрана, заполненный единицами (максимасимальная z-координата после преобразования пирамиды зрения в куб). Далее для каждого треугольника мы делаем *растеризацию* – определяем пиксели, принадлежащие треугольнику. Для этого мы пересекаем треугольник горизонтальными линиями и определяем пиксели, соответствующие крайним точкам в пересечении (пиксели M, N на рис. 6) – это и будет отрезок пикселей, накрываемых данным треугольником. Для каждого из накрываемых пикселей мы вычисляем z-координату следующим образом: сначала мы линейно интерполируем z-кооринату для крайних точек, после чего линейно интерполируем полученные значения (например, на рис. 6: $z_M = z_A \cdot \frac{|AM|}{|AB|} + z_B \cdot \frac{|BM|}{|AB|}$, $z_N = z_B \cdot \frac{|BN|}{|BC|} + z_C \cdot \frac{|NC|}{|BC|}$, $z_Q = z_M \cdot \frac{|MQ|}{|MN|} + z_N \cdot \frac{|NQ|}{|MN|}$). Если посчитанное значение меньше, чем в Z-буффере, то мы обновляем последний (напомню, что чем меньше z, тем ближе точка к экрану). Т.о. для каждого пикселя на экране мы запомним треугольник, расположенный наиболее близко к камере в данном пикселе.

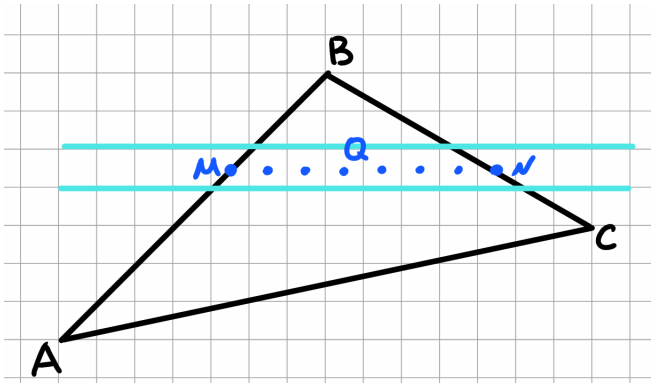


Рис. 6: Итерация растеризации треугольника

4.7 Интерполяция скалярных величин

Последним этапом при отрисовке пикселей треугольника является определение цвета пикселя. Для этого нам нужно научиться интерполировать информацию, записанную в вершинах треугольника. Это можно делать также, как мы интерполировали z-координату в алгоритме растеризации, но здесь есть важная деталь. При растеризации мы работаем с проекцией треугольника на экране, а значит равные шаги на проекции не обязательно соответствуют равным изменениям величин треугольника:

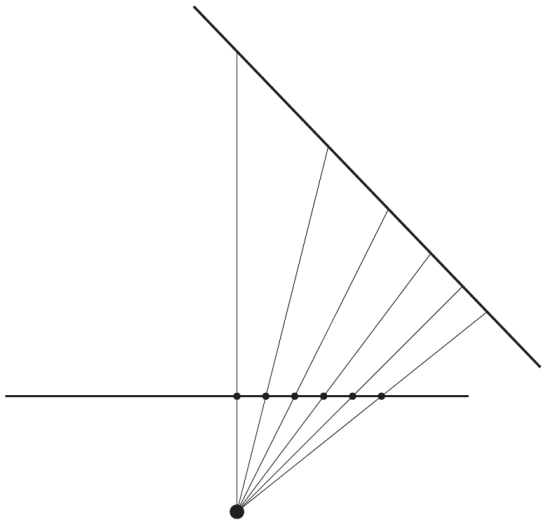


Рис. 7: Равные шаги по проекции соответствуют разным шагам по поверхности треугольника

Это не является проблемой в алгоритме Z-буффера из-за свойств преобразования $\mathbf{M_{frustum}}$ (подробнее чуть ниже), но это станет проблемой для других неподготовленных величин (например, если наивно интерполировать текстурные координаты, то полученная текстура будет "растягиваться"при отдалении точки от камеры).

Решение этой проблемы несложное: для точки A из однородного пространства куба зрения обозначим

- p_A – координаты проекции A на экране (просто координаты x, y точки A)
- b_A – интерполируемая скалярная величина
- z_A – z-координату точки A **до преобразования пирамиды зрения**.

Тогда если для точек A, B, C выполнено, что $p_C = (1-t) \cdot p_A + t \cdot p_B$, то

$$\frac{b_C}{z_C} = (1-t) \cdot \frac{b_A}{z_A} + t \cdot \frac{b_B}{z_B} \quad (*)$$ [2]

Т.о. нам достаточно линейно интерполировать значение b/z, а потом домножить полученное значение на z. Но тогда почему в алгоритме Z-буффера мы интерполируем z линейно? Идея в том, что преобразование $\mathbf{M_{frusutm}}$ действует на z обратно пропорционально: $z' = \frac{p}{z} + q$, $p, q \in \mathbb{R}$. Значит, если мы запишем линейную интерполяцию z' для точки C, мы получим:

$$z'_C = (1-t) \cdot z'_A + t \cdot z'_B \Leftrightarrow \frac{p}{z_C} + q = (1-t) \cdot \left(\frac{p}{z_A} + q\right) + t \cdot \left(\frac{p}{z_B} + q\right) \Leftrightarrow \frac{p}{z_C} = (1-t) \cdot \frac{p}{z_A} + t \cdot \frac{p}{z_B}$$

А самое правое выражение – это просто $(*)$, в котором положили $b \equiv p$. Именно поэтому линейная интерполяция в алгоритме Z-буффера корректна.

5 Архитектура приложения

5.1 Ключевые классы

- **Application** – ответственен за работу приложения: создает окно, инициализирует ресурсы приложения, обрабатывает пользовательский ввод, вызывает отрисовку изображения и выводит отрисованное изображение на экран.
  - `Конструктор`: создает ключевые объекты рендера (`Renderer`, `Camera`, `World`) и окно приложения, загружает объекты сцены.
  - `run()`: запускает run-time loop приложения: [3]

```
while (render_window_.isOpen()) {
    FrameMovement camera_movement = calculateCameraMovementByInputs();
    camera_.moveCamera(camera_movement);
    Image image = renderer_.renderImage(camera_, world_);
    drawImageInWindow(image);
    closeWindowIfAsked();
}
```

- **World** – ответственен за хранение объектов сцены.
  - `addObject(Object&& object)`: добавляет объект в сцену.
  - `getTriangles()`: возвращает список полигонов всех объектов сцены. Для избежания возможных лишних копирований данный метод возвращает легковесный прокси-контейнер `Triangles`, предоставляющий методы `begin()`, `end()` для итерации по треугольникам.
- **Camera** – ответственен за работу с камерой во время отрисовки изображения.
  - `Конструктор`: создает матрицу преобразования пирамиды зрения `transform_mat_` ($\mathbf{M_{frustum}}$ из разд. 4.5), плоскости, ограничивающие пирамиду зрения `clipping_planes_` (см. разд. 4.4), и матрицу `movement_mat`, о которой чуть подробнее.

    `movement_mat` переводит точку из мирового пространства в пространство камеры, при том что камера считается неподвижной и расположенной как на рис. 11. Поэтому данная матрица является **обратной** к реальному перемещению камеры (ведь если зафиксировать камеру, то объекты будут двигаться вокруг нее в обратном направлении).
  - `transformWorldTriangles(const World& world)`: преобразует треугольники сцены в однородное пространство камеры. Полученные треугольники возвращаются в формате, подготовленном к алгоритму Z-буффера – `HTriangle` (подробнее в разд. 5.2).

    Данное преобразование состоит из нескольких этапов:

[2] Обоснование данного свойства можно найти в [10], sec. 5.4

[3] Здесь и далее приведенный код имеет иллюстративный характер и не обязательно совпадает с реальным кодом приложения

```
Triangle shifted_triangle = moveTriangleToCameraSpace(triangle);
std::vector<Triangle> clipped_triangles = clipTriangle(shifted_triangle);
for (const Triangle& clipped_triangle : clipped_triangles) {
    result.emplace_back(
        transformVertex(clipped_triangle.vertices[0]),
        transformVertex(clipped_triangle.vertices[1]),
        transformVertex(clipped_triangle.vertices[2]));
}
```

  * `moveTriangleToCameraSpace(triangle)`: умножает вершины треугольника на `movement_mat` (первый этап разд. 4.2)
  * `clipTriangle(shifted_triangle)`: делает clipping с пирамидой зрения, описанный в разд. 4.4. Обращу внимание, что данный метод возвращает список треугольников, а не один.
  * `transformVertex(const Vertex& vertex)`: умножает вершину треугольника на матрицу `transform_mat_` и переводит ее в HVertex (см. разд. 5.2).

- `moveCamera(FrameMovement movement)`: вычисляет матрицу M, соответсвующую сдвигу камеры в течение кадра, и умножает `movement_mat` на M^{-1}. Обращу внимание, что данный метод не меняет матрицу `transform_mat_` и плоскости `clipping_planes_`, т.к. камера считается неподвижной в своем пространстве.

- **Renderer** – ответственен за создание изображения.

  - `renderImage(const Camera& camera, const World& world)` – создает изображение объектов мира `world` на основании текущего положения камеры `camera`.

    Данный метод также состоит из нескольких этапов:

```
Image Renderer::renderImage(const Camera& camera, const World& world) {
    return createImage(constructZBuffer(createHTriangles(camera, world)));
}
```

    * `createHTriangles(camera, world)`: вызывает `camera.transformWorldTriangles(world)`.
    * `constructZBuffer(h_triangles)`: выполняет алгоритм Z-буффера (разд. 4.6). Отмечу, что во время растеризации треугольника для каждого пикселя сразу же вычисляется его цвет. Тем самым после данного алгоритма отпадает нужда в хранении треугольников – вся необходимая информация для создания изображения уже сохранена.
    * `createImage(z_buffer)`: проходится по Z-буфферу и создает картинку в формате, понятном SFML.

5.2 Вспомогательные классы

- **Vertex** – вершина треугольника. Хранит позицию в пространстве и скалярные атрибуты, использующиеся для определения цвета пикселей.

  Атрибуты вершины вынесены в отдельную структуру `VertexAttributes`, предоставляющие методы для линейной интерполяции и вычисления цвета. Это преследует две цели: во-первых, это упрощает добавление других атрибутов (например, векторов нормали), во-вторых, это избавляет от дублирования кода в `HVertex`.

- **Triangle** – сам треугольник. Просто хранит 3 вершины `Vertex`.

- **HVertex** – вершина треугольника, переведенная камерой в однородное пространство (отсюда $h = homogeneous$ в названии). По сути является тем же `Vertex`, только подготовленным к алгоритму Z-буффера.

  `HVertex`, как и `Vertex`, хранит позицию вершины в пространстве и скалярные атрибуты вершины, но также она хранит величину `z_reciprocal_`. Идея в следующем: как мы знаем из разд. 4.7, во время растеризации мы не имеем право наивно интерполировать скалярную величину b – вместо этого мы должны линейно интерполировать b/z, где z – z-координата вершины до преобразований. Поэтому во

время преобразований в `Camera` мы запоминаем `z_reciprocal_` $= 1/z$, после используем это значение во время линейной интерполяции в соответствии с формулой (*). Само же значение `z_reciprocal_` мы интерполируем линейно.

- **HTriangle** – аналог `Triangle`, только в однородном пространстве камеры. Он также просто хранит 3 `HVertex`, только с 1 отличием: вершины отсортированы в порядке убывания y-координаты. Сделано это сугубо для удобства растеризации.

6 Демонстрация результатов

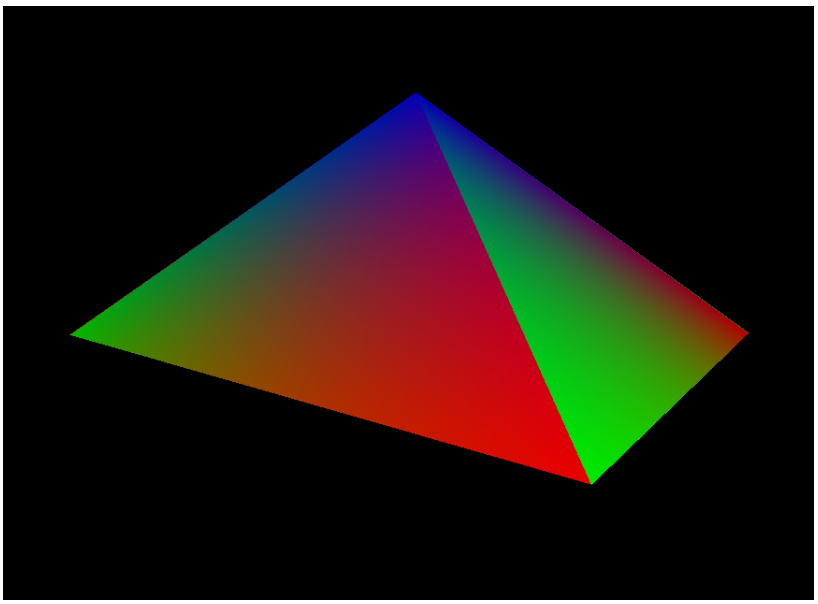


Рис. 8: Демонстрация интерполяции скалярных величин (градиент на сторонах создан путем интерполяции цветов в вершинах)

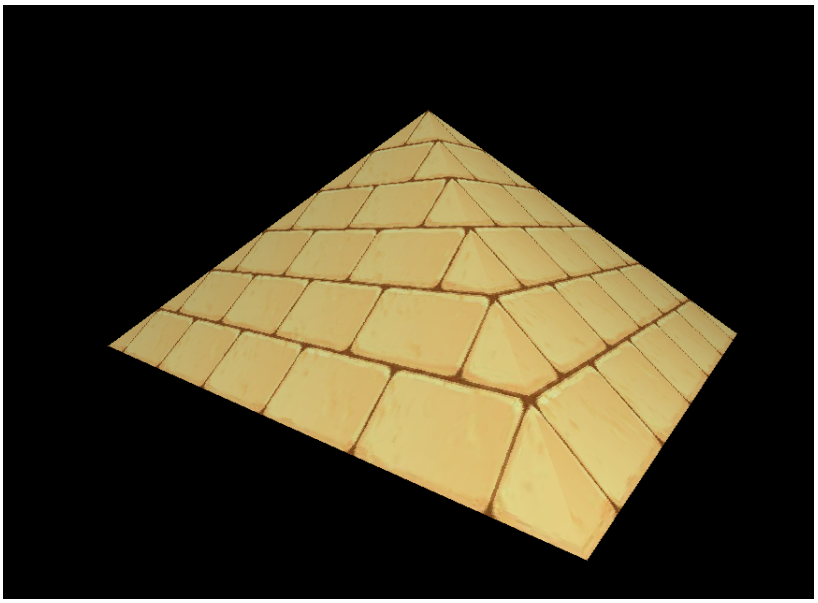


Рис. 9: Демонстрация работы с текстурами

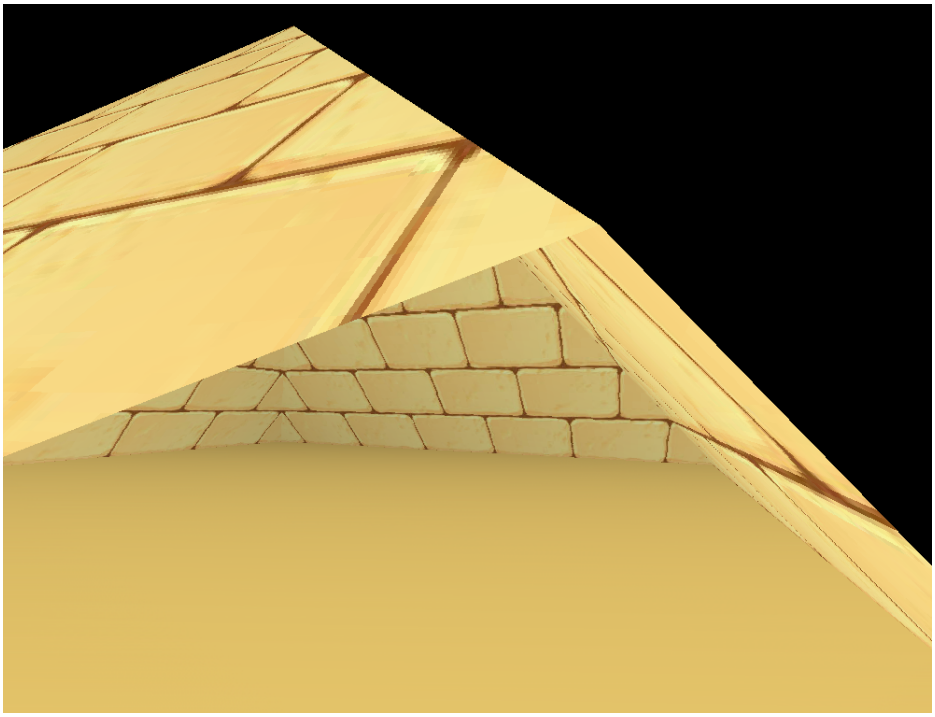


Рис. 10: Демонстрация работы клиппинга

Немного упомяну про производительность приложения на примере сцены с пирамидой. Частота кадров зависит от близости пирамиды к камере и варьируется от 80 кадров / сек. вдали от пирамиды до 17 кадров / сек. вблизи. Данная зависимость объясняется количеством пикселей, для которых приходится проводить расчеты (чем ближе пирамида, тем большую часть экрана она накрывает). При заезде внутрь пирамиды производительность не падает еще ниже и остается в районе 20 кадров / сек.

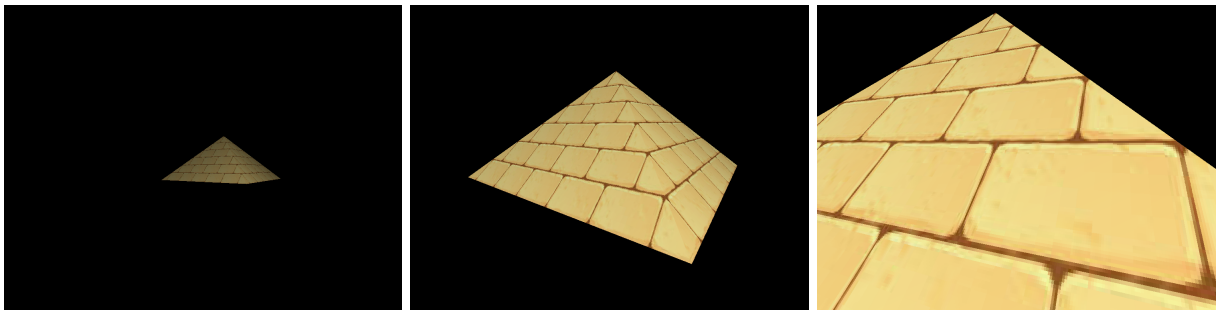


Рис. 11: Частота кадров зависит от расстояния между пирамидой и камерой. В приведенных позициях камеры FPS составляет (слева направо): 80, 50, 17

Список литературы

[1] Clang format. URL: https://clang.llvm.org/docs/ClangFormat.html.

[2] Cmake. URL: https://cmake.org/.

[3] Eigen library. URL: https://eigen.tuxfamily.org/index.php?title=Main_Page.

[4] Gcc compiler. URL: https://gcc.gnu.org/.

[5] Github version control. URL: https://github.com/.

[6] Google style guide. URL: https://google.github.io/styleguide/cppguide.html.

[7] Sfml library. URL: https://www.sfml-dev.org/.

[8] Sutherland–hodgman algorithm wikipedia article. URL: https://en.wikipedia.org/wiki/Sutherland%E2%80%93Hodgman_algorithm.

[9] Vs code editor. URL: https://code.visualstudio.com/.

[10] Eric Lengyel. *Mathematics for 3D Game Programming and Computer Graphics, Third Edition.* Course Technology PTR, 2011.

[11] Трушин Д. В. Консультация к проекту «3d render с нуля», часть 1. 2021. URL: https://disk.yandex.ru/d/vfkRg0Q6np0W6w/%D0%9A%D0%BE%D0%BD%D1%81%D1%83%D0%BB%D1%8C%D1%82%D0%B0%D1%86%D0%B8%D1%8F%2020.10.2021/3DRenderer%20part%201.mp4.

[12] Трушин Д. В. Консультация к проекту «3d render с нуля», часть 1. 2021. URL: https://disk.yandex.ru/d/vfkRg0Q6np0W6w/%D0%9A%D0%BE%D0%BD%D1%81%D1%83%D0%BB%D1%8C%D1%82%D0%B0%D1%86%D0%B8%D1%8F%2020.10.2021/3DRenderer%20part%202.mp4.